

Logical Operators

Combining Conditions for Complex Choices

Logical operators allow your program to connect multiple boolean statements together. They are the building blocks of **complex decision making**.

&&

AND

Both must be true

||

OR

One must be true

!

NOT

Reverse the state

1. Usage Reference

Op	Comparison Type	How it works
&&	Logical AND	Returns true if both statements are true.
	Logical OR	Returns true if at least one statement is true.
!	Logical NOT	Turns true into false (and vice versa).

2. Real-Life Example: Login Logic

A typical login system checks if a user is logged in AND whether they have specific permissions.

```
boolean isLoggedIn = true;
boolean isAdmin = false;

// Check for a regular user (Logged in but NOT an Admin)
System.out.println(isLoggedIn && !isAdmin); // Result: true

// Access allowed if Logged in OR is Admin
System.out.println(isLoggedIn || isAdmin); // Result: true

// Check if NOT logged in
System.out.println(!isLoggedIn); // Result: false
```

Key Takeaways

- **Decision Chains:** Use `&&` and `||` to link multiple comparison operators together (e.g., `x > 0 && x < 100`).
- **Reversal:** Use `!` to check for negative conditions (e.g., "is not empty").
- **Efficiency:** Java uses "short-circuiting"—if the first part of an `&&` is false, it won't even check the second part!
- **Parentheses:** Always use parentheses when mixing `&&` and `||` to ensure your logic runs in the right order.